# Lecture 3: From Vectors to Entanglement – Math Foundations & Quantum Communication 🚀

*Bridging Linear Algebra Review with Quantum Protocols*

---

## Lecture Overview 📋

1. **Math Review:** Vectors, inner products, eigenvalues, Hermitian matrices, tensor products 🧮
2. **Dirac Notation Deep Dive:** Bras, kets, and operators 📐
3. **Composite Systems:** Building multi-qubit states 🧱
4. **Entanglement:** The "spooky" correlation explained mathematically 👻
5. **Bell States:** Maximally entangled states 🔔
6. **Quantum Communication Protocols:** 📡
    - Superdense coding
    - Quantum teleportation
7. **Quantum Key Distribution (BB84):** Real-world application 🔐
8. **PennyLane Demos:** Creating and measuring Bell states 💻

---

## Part 1: Linear Algebra Refresher (with Quantum Flavor) 🧮

### 1.1 Vectors in Quantum Mechanics 🔹

A **vector** in quantum mechanics represents a state. For a single qubit, we work in $\mathbb{C}^2$ – the space of complex 2-dimensional vectors.

**Column vector notation (ket):**

$$|\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \quad \alpha, \beta \in \mathbb{C}$$

---

**Row vector notation (bra):**

$$\langle\psi| = \begin{bmatrix} \alpha^* & \beta^* \end{bmatrix}$$

where $\alpha^*$ is the complex conjugate of $\alpha$.

---

**Example:**

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \langle 0| = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

---

$$|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \langle 1| = \begin{bmatrix} 0 & 1 \end{bmatrix}$$

---

### 1.1.1 Complex Numbers: The Language of Quantum Mechanics 🔢

Before diving deeper into vectors and inner products, we need to be comfortable with **complex numbers** – the alphabet of quantum theory.

---

## What is a Complex Number?

A complex number $z$ is written as:

$$z = a + ib, \quad a, b \in \mathbb{R},\ i = \sqrt{-1}$$

- $a$ = **real part** $\mathrm{Re}(z)$
- $b$ = **imaginary part** $\mathrm{Im}(z)$

## The Complex Plane 🌌

We can visualise $z$ as a point in a 2D plane:

- Horizontal axis: real part
- Vertical axis: imaginary part

```
       Im
        ↑
        |   • z = a + i b
        |  /|
        | / |
        |/θ |
────────┼────────→ Re
        |   |
        |   |
        |   • z* = a - i b
        ↓
```

## Modulus (Amplitude) and Phase 📐

Every complex number has a **modulus** (or **amplitude**) $r$ and a **phase** (or **argument**) $\theta$:

$$r = |z| = \sqrt{a^2 + b^2}, \qquad \theta = \arg(z) = \tan^{-1}\left(\frac{b}{a}\right)$$

**Polar form:**

$$z = re^{i\theta} = r(\cos\theta + i\sin\theta)$$

- $r$ tells us the "size" (always non-negative)
- $\theta$ tells us the "angle" (usually in radians)

## Complex Conjugate – Mirror Across the Real Axis 🔄

The **complex conjugate** of $z = a + ib$ is:

$$z^* = a - ib$$

In polar form:

$$z = re^{i\theta} \quad \Longrightarrow \quad z^* = re^{-i\theta}$$

Geometrically, it's the reflection of $z$ across the real axis.

---

## Why Do We Care? 💡

1. **Probabilities come from squared magnitudes:**
   If a quantum amplitude is $z$, the probability is $|z|^2 = z^* z$.
   (E.g., $|\alpha|^2 + |\beta|^2 = 1$ for a qubit state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$.)
2. **Inner products involve conjugates:**
   $\langle\phi|\psi\rangle = \phi_0^*\psi_0 + \phi_1^*\psi_1$ – the conjugate of the left vector's components appears.
3. **Hermitian conjugates (†) are conjugate transposes:**
   $(|\psi\rangle)^\dagger = \langle\psi|$, and for matrices $(U)^\dagger = (U^T)^*$.

---

## Quick Check: Conjugate in Action ⚡

```
import numpy as np

z = 3 + 4j                # 3 + 4i
print(f"z = {z}")
print(f"Real part: {z.real}, Imag part: {z.imag}")
print(f"Modulus: {abs(z):.2f}")
print(f"Conjugate: {z.conj()}")
print(f"z * z* = {z * z.conj()} = |z|^2 = {abs(z)**2}")
```

---

Output:

```
z = (3+4j)
Real part: 3.0, Imag part: 4.0
Modulus: 5.00
Conjugate: (3-4j)
z * z* = (25+0j) = |z|^2 = 25.0
```

---

**Key takeaway:** Complex numbers are not just a mathematical convenience – they are essential for the interference effects that make quantum mechanics so powerful. The conjugate is your best friend when computing probabilities and inner products! 🌟

---

## 1.2 Inner Product (Dot Product for Complex Vectors) 📏

The **inner product** $\langle\phi|\psi\rangle$ tells us how "similar" two states are.

---

**Definition:**

$$\langle\phi|\psi\rangle = \phi_0^*\psi_0 + \phi_1^*\psi_1$$

---

**Properties:**

- $\langle\psi|\psi\rangle = |\alpha|^2 + |\beta|^2 = 1$ (normalization) ✅
- $\langle\phi|\psi\rangle = \langle\psi|\phi\rangle^*$ (conjugate symmetry) 🔄
- If $\langle\phi|\psi\rangle = 0$, states are **orthogonal** ○

---

**Examples from HW1 (A.2):**

```python
import numpy as np

# Define states
zero = np.array([1, 0], dtype=complex)
one = np.array([0, 1], dtype=complex)
plus = (zero + one) / np.sqrt(2)
minus = (zero - one) / np.sqrt(2)

# Inner products
print(f"⟨0|1⟩ = {np.vdot(zero, one)}")        # 0 (orthogonal)
print(f"⟨+|+⟩ = {np.vdot(plus, plus)}")        # 1 (normalized)
print(f"⟨+|-⟩ = {np.vdot(plus, minus)}")        # 0 (orthogonal)
```

---

## 1.3 Outer Product and Projectors 🎯

The **outer product** $|\psi\rangle\langle\phi|$ creates an operator (matrix).

---

**Example:** Projector onto $|0\rangle$:

$$|0\rangle\langle 0| = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

---

**Why this matters:** Measurement operators are projectors! (Postulate 3) 💡

---

## 1.4 Matrices as Operators 🎛️

Quantum gates are **matrices** that act on state vectors:

$$U|\psi\rangle = (\text{matrix}) \times (\text{vector})$$

---

**Pauli Matrices** (from HW1 A.5):

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

---

## 1.5 Hermitian Conjugate (†) 🔄

The **Hermitian conjugate** (or adjoint) of a matrix is the **conjugate transpose**:

- For vectors: $(|\psi\rangle)^\dagger = \langle\psi|$
- For matrices: $(U)^\dagger = (U^T)^*$

---

### A matrix is Hermitian if $M^\dagger = M$

From HW1 A.9: Pauli matrices are Hermitian:

```
X = np.array([[0,1],[1,0]], dtype=complex)
print(np.allclose(X, X.conj().T))  # True
```

---

**Why Hermitian matters:** All measurable observables (like energy, spin) are represented by Hermitian operators. Their eigenvalues are **real** numbers – the possible measurement outcomes! 📊

---

## 1.6 Eigenvalues and Eigenvectors 🔑

For a matrix $M$, if $M|v\rangle = \lambda|v\rangle$, then:

- $\lambda$ is an **eigenvalue** (measurement outcome)
- $|v\rangle$ is an **eigenvector** (state after measurement)

---

**From HW1 A.6:** For Pauli Z:

$$Z|0\rangle = (+1)|0\rangle, \quad Z|1\rangle = (-1)|1\rangle$$

So eigenvalues are $+1$ and $-1$, eigenvectors are $|0\rangle$ and $|1\rangle$.

---

**Key insight:** When you measure an observable, you always get one of its eigenvalues! 🎲

---

## 1.7 Unitary Matrices 🔄

A matrix $U$ is **unitary** if $U^\dagger U = UU^\dagger = I$.

---

**Properties:**

- Preserves inner products: $\langle U\phi|U\psi\rangle = \langle\phi|\psi\rangle$
- Preserves normalization: $||U|\psi\rangle|| = 1$
- All quantum gates are unitary! (Postulate 2) ✅

---

**From HW1 A.9:** Check unitarity:

```
print(np.allclose(X @ X.conj().T, np.eye(2)))  # True
```

---

## 1.8 Tensor Product – The Most Important Operation for Multiple Qubits 🧩

The **tensor product** $\otimes$ combines systems:

---

**For vectors:** If $|a\rangle \in \mathbb{C}^m$ and $|b\rangle \in \mathbb{C}^n$, then $|a\rangle \otimes |b\rangle \in \mathbb{C}^{m\times n}$

---

**Example:** Two qubits:

$$|0\rangle \otimes |0\rangle = \begin{bmatrix}1\\0\end{bmatrix} \otimes \begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}1\\0\\0\\0\end{bmatrix} = |00\rangle$$

---

**For matrices:** $A \otimes B$ means apply $A$ to first system, $B$ to second.

---

**Python demo:**

```
# Tensor product in NumPy
zero = np.array([1, 0])
one = np.array([0, 1])

# Create |01⟩ = |0⟩ ⊗ |1⟩
zero_one = np.kron(zero, one)
print(zero_one)  # [0 1 0 0] (indexing: 00,01,10,11)

# Tensor product of Pauli X and I
X = np.array([[0,1],[1,0]])
I = np.eye(2)
X_I = np.kron(X, I)  # 4x4 matrix
```

---

## 1.9 Why All This Math Matters for Quantum Communication 💡

| Math Concept | Physical Meaning | Used In |
|---|---|---|
| Vector $\mid \psi\rangle$ | Quantum state | All protocols |
| Inner product $\langle\phi \mid \psi\rangle$ | Overlap/Probability amplitude | Measurement |
| Eigenvalues | Possible measurement outcomes | BB84 basis choice |
| Hermitian $M^\dagger = M$ | Observable quantities | All measurements |
| Unitary $U^\dagger U = I$ | Quantum gates (evolution) | Creating Bell states |
| Tensor product $\otimes$ | Multiple qubits | Entanglement |

---

# Part 2: Composite Systems & Tensor Products 🧩

## 2.1 Two-Qubit Basis States 🔢

For two qubits, the computational basis consists of **four** states:

$$|00\rangle = \begin{bmatrix}1\\0\\0\\0\end{bmatrix}, \quad |01\rangle = \begin{bmatrix}0\\1\\0\\0\end{bmatrix}, \quad |10\rangle = \begin{bmatrix}0\\0\\1\\0\end{bmatrix}, \quad |11\rangle = \begin{bmatrix}0\\0\\0\\1\end{bmatrix}$$

---

**General two-qubit state:**

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle$$

with $\sum |\alpha_{ij}|^2 = 1$.

---

## 2.2 Product States vs. Entangled States 🔗

**Product state:** Can be written as $(a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle)$

---

Example: $|+0\rangle = |+\rangle \otimes |0\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$

---

**Entangled state:** Cannot be factored into separate single-qubit states

---

Example: $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$

---

**Check if entangled:** Try to find $a, b, c, d$ such that:

$$\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = (a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle)$$

---

This would require $ac = 1/\sqrt{2}$, $bd = 1/\sqrt{2}$, and $ad = bc = 0$ – impossible! (If $a = 0$, then $ac = 0$; if $b = 0$, then $bd = 0$.) ✖

---

## 2.3 Creating Entanglement with CNOT 🔧

The **CNOT** (controlled-NOT) gate is the key to creating entanglement:

```
q0: ───@───
       |
q1: ───X───
```

---

**Matrix form** (control = q0, target = q1):

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

**Action:** If control is $|1\rangle$, flip target; otherwise do nothing.

---

**Creating a Bell state:**

```
|0⟩ ──H──@──
         |
|0⟩ ─────X──
```

---

**Step-by-step:**

1. Start: $|00\rangle$
2. H on first qubit: $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$
3. CNOT:
   - $|00\rangle \to |00\rangle$ (control 0, no flip)
   - $|10\rangle \to |11\rangle$ (control 1, flip target)
4. Result: $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ – the Bell state $|\Phi^+\rangle$! 🎉

---

## 2.4 The Four Bell States 🔔

The Bell states are maximally entangled two-qubit states:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$$

---

**Key properties:**

- Maximally entangled (any measurement on one qubit gives random result) 🎲
- Perfectly correlated (for $|\Phi^+\rangle$, measuring both qubits in Z basis gives same result) 🤝
- Orthogonal: $\langle\Phi^+|\Phi^-\rangle = 0$, etc.

---

**Circuit to create all Bell states:**

| Start state | After H on q0 + CNOT | Bell state |
|---|---|---|
| $\mid 00\rangle$ | $\frac{1}{\sqrt{2}}(\mid 00\rangle + \mid 11\rangle)$ | $\mid \Phi^+\rangle$ |
| $\mid 01\rangle$ | $\frac{1}{\sqrt{2}}(\mid 01\rangle + \mid 10\rangle)$ | $\mid \Psi^+\rangle$ |
| $\mid 10\rangle$ | $\frac{1}{\sqrt{2}}(\mid 00\rangle - \mid 11\rangle)$ | $\mid \Phi^-\rangle$ |
| $\mid 11\rangle$ | $\frac{1}{\sqrt{2}}(\mid 01\rangle - \mid 10\rangle)$ | $\mid \Psi^-\rangle$ |

---

# Part 3: Quantum Communication Protocols 📡

## 3.1 Superdense Coding – Send 2 Classical Bits Using 1 Qubit ✨

**The magic:** Alice can send Bob **two classical bits** by transmitting only **one qubit**, if they already share an entangled pair! 🪄

---

**Setup:**

1. Create Bell pair $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$
2. Give one qubit to Alice, one to Bob

---

**Protocol:**

| Alice's message (2 bits) | Alice's operation on her qubit | Resulting Bell state |
|---|---|---|
| 00 | I (do nothing) | $\|\Phi^+\rangle = \frac{1}{\sqrt{2}}(\|00\rangle + \|11\rangle)$ |
| 01 | X (bit flip) | $\|\Psi^+\rangle = \frac{1}{\sqrt{2}}(\|01\rangle + \|10\rangle)$ |
| 10 | Z (phase flip) | $\|\Phi^-\rangle = \frac{1}{\sqrt{2}}(\|00\rangle - \|11\rangle)$ |
| 11 | iY = XZ (both) | $\|\Psi^-\rangle = \frac{1}{\sqrt{2}}(\|01\rangle - \|10\rangle)$ |

---

Alice then sends her qubit to Bob. Bob now has **both qubits** and can perform a **Bell**

**measurement** to determine which of the four states he has, recovering the 2 classical bits.

---

**Bell measurement circuit:**

```
q0 (Alice's) ────@──H── Measure
                 |
q1 (Bob's) ──────X─── Measure
```

---

**Why it works:** The CNOT + H gate transforms Bell states back to computational basis:

$$|\Phi^+\rangle \to |00\rangle, \quad |\Phi^-\rangle \to |10\rangle, \quad |\Psi^+\rangle \to |01\rangle, \quad |\Psi^-\rangle \to |11\rangle$$

---

**Key insight:** The entangled pair acts as a "resource" that enables sending 2 bits with 1 qubit! 💎

---

## 3.2 Quantum Teleportation – Send an Unknown Qubit State 📤

**The magic:** Alice can send an unknown quantum state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ to Bob using only **classical communication** and a shared entangled pair! 📦

---

**Setup:**

1. Create Bell pair $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$
2. Give one qubit to Alice (qubit B), one to Bob (qubit C)
3. Alice has the unknown state $|\psi\rangle$ on qubit A

---

**Initial state:** $|\psi\rangle_A \otimes |\Phi^+\rangle_{BC}$

---

**Step-by-step:**

1. **Alice applies CNOT** (control = qubit A, target = qubit B) 🔀

---

2. **Alice applies H** to qubit A 🧲

---

3. **Alice measures** both qubits A and B (2 classical bits) 📏

---

4. **Alice sends** these 2 bits to Bob 📨

---

5. **Bob applies corrections** based on Alice's bits:

---

- If bits = 00: do nothing
- If bits = 01: apply X
- If bits = 10: apply Z
- If bits = 11: apply X then Z

---

**Result:** Bob's qubit C becomes exactly $|\psi\rangle$! 🎯

---

**Mathematical outline:**

Initial: $(\alpha|0\rangle_A + \beta|1\rangle_A) \otimes \frac{1}{\sqrt{2}}(|0_B 0_C\rangle + |1_B 1_C\rangle)$

After CNOT and H on A, the state becomes:

$$\frac{1}{2}[|00\rangle_{AB}(\alpha|0\rangle_C + \beta|1\rangle_C) + |01\rangle_{AB}(\alpha|1\rangle_C + \beta|0\rangle_C) + |10\rangle_{AB}(\alpha|0\rangle_C - \beta|1\rangle_C) + |11\rangle_{AB}(-\alpha|$$

Depending on Alice's measurement outcome, Bob's qubit is in one of four states, each related to $|\psi\rangle$ by a known Pauli operation.

---

## 3.3 Comparison: Superdense Coding vs. Teleportation ⚖️

| Protocol | What's sent | What's transmitted | Resource |
|---|---|---|---|
| Superdense coding | 2 classical bits | 1 qubit | Shared entanglement |
| Teleportation | 1 qubit (unknown) | 2 classical bits | Shared entanglement |

**Common theme:** Entanglement + classical communication = quantum information processing! 🔄

---

# Part 4: Quantum Key Distribution (BB84) 🔐

## 4.1 The Problem: Secure Communication 🕵️

**Classical problem:** How can Alice and Bob share a secret key without Eve eavesdropping?

---

**Classical solution:** Public key cryptography (RSA) – based on mathematical difficulty (factoring) 🔢

---

**Quantum solution:** BB84 – based on laws of physics (no-cloning, measurement disturbance) ⚛️

## 4.2 The BB84 Protocol 📋

**Goal:** Generate a shared secret key between Alice and Bob

---

**Setup:**

- Alice can prepare qubits in one of **four states**:
  - **Z basis:** $|0\rangle$ (bit 0), $|1\rangle$ (bit 1)
  - **X basis:** $|+\rangle$ (bit 0), $|-\rangle$ (bit 1)
- Bob can measure in either **Z basis** or **X basis**

---

**Step 1: Quantum Transmission** 📤

For each qubit (say N=100 rounds):

---

1. **Alice randomly chooses:**
   - Basis: Z or X (with 50% probability each) 🎲
   - Bit: 0 or 1 (with 50% probability each)

---

2. **Alice prepares** the corresponding state:
   - Z/0 → $|0\rangle$, Z/1 → $|1\rangle$
   - X/0 → $|+\rangle$, X/1 → $|-\rangle$

---

3. **Alice sends** the qubit to Bob

---

4. **Bob randomly chooses** basis: Z or X 🎲

---

5. **Bob measures** in his chosen basis

---

**Step 2: Classical Communication (over public channel)** 📢

1. **Basis revelation:** Alice and Bob announce which bases they used **(but NOT the bits!**)

---

2. **Sifting:** They keep only rounds where they used the **same basis**
    - If bases match, Bob's measurement result should match Alice's bit (ideally)
    - If bases differ, they discard that round (results are random/unrelated)

After sifting, they have a raw key (about half the rounds, ~50 bits). ✂

---

**Step 3: Detecting Eavesdropping** 🕵♀

Eve doesn't know which basis Alice used. If Eve intercepts and measures:

---

- If Eve guesses basis correctly ✅
    - → she resends correct state
    - → no error

---

- If Eve guesses wrong basis ❌
    - → she collapses the state

- → when Bob measures in Alice's basis, he gets random result with 50% error probability

---

**How to detect Eve?**

---

1. Alice and Bob publicly compare a **random subset** of their raw key bits (say 20 bits) 🔍

---

2. If more than a small threshold (e.g., 0%) differ, they know Eve was listening
   - → abort 🚫

---

## 4.3 Why BB84 Works: The No-Cloning Theorem 🚫🧬

**No-cloning theorem:** It's impossible to make a perfect copy of an unknown quantum state.

---

**Proof sketch:** If a cloning machine existed, it would have to map:

- $|0\rangle|e\rangle \to |0\rangle|0\rangle$
- $|1\rangle|e\rangle \to |1\rangle|1\rangle$

---

But then for $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$:
$|+\rangle|e\rangle \to \frac{1}{\sqrt{2}}(|0\rangle|0\rangle + |1\rangle|1\rangle) \neq |+\rangle|+\rangle$ (which would be $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$)

---

**Implication for BB84:** Eve cannot simply copy the qubit and measure later – she must measure

now, potentially disturbing it! ⚠

---

## 4.4 Simple Python Simulation of BB84 🐍

```python
import numpy as np
import random

def bb84_simulation(n_qubits=100, eavesdrop=False):
    """Simulate BB84 protocol with optional eavesdropping"""

    # Basis mapping
    bases = ['Z', 'X']
    # States: for basis Z: 0->|0>, 1->|1>; for basis X: 0->|+>, 1->|->

    alice_bases = []
    alice_bits = []
    bob_bases = []
    bob_results = []
    eve_bases = [] if eavesdrop else None
    eve_results = [] if eavesdrop else None

    for _ in range(n_qubits):
        # Alice's choices
        basis_a = random.choice(bases)
        bit_a = random.randint(0, 1)
        alice_bases.append(basis_a)
        alice_bits.append(bit_a)

        # Determine quantum state
        if basis_a == 'Z':
            # |0> or |1>
            state = np.array([1, 0]) if bit_a == 0 else np.array([0, 1])
        else:  # X basis
            # |+> = (|0>+|1>)/√2, |-> = (|0>-|1>)/√2
            if bit_a == 0:
                state = np.array([1/np.sqrt(2), 1/np.sqrt(2)])
            else:
                state = np.array([1/np.sqrt(2), -1/np.sqrt(2)])

        # Eve intercepts (if eavesdropping)
        if eavesdrop:
            basis_e = random.choice(bases)
            eve_bases.append(basis_e)
```

```
        # Eve measures
        if basis_e == 'Z':
            prob0 = abs(state[0])**2
            result_e = 0 if random.random() < prob0 else 1
            # Collapse state
            if result_e == 0:
                state = np.array([1, 0])
            else:
                state = np.array([0, 1])
        else:  # X basis
            # Convert to X basis probabilities
            plus_amplitude = (state[0] + state[1])/np.sqrt(2)
            prob_plus = abs(plus_amplitude)**2
            result_e = 0 if random.random() < prob_plus else 1
            # Collapse to |+> or |->
            if result_e == 0:
                state = np.array([1/np.sqrt(2), 1/np.sqrt(2)])
            else:
                state = np.array([1/np.sqrt(2), -1/np.sqrt(2)])

        eve_results.append(result_e)

    # Bob measures
    basis_b = random.choice(bases)
    bob_bases.append(basis_b)

    if basis_b == 'Z':
        prob0 = abs(state[0])**2
        result_b = 0 if random.random() < prob0 else 1
    else:  # X basis
        plus_amplitude = (state[0] + state[1])/np.sqrt(2)
        prob_plus = abs(plus_amplitude)**2
        result_b = 0 if random.random() < prob_plus else 1

    bob_results.append(result_b)

# Sifting: keep only where bases match
sifted_alice = []
```

```
    sifted_bob = []
    for i in range(n_qubits):
        if alice_bases[i] == bob_bases[i]:
            sifted_alice.append(alice_bits[i])
            sifted_bob.append(bob_results[i])

    # Check error rate on a subset
    n_check = min(20, len(sifted_alice)//2)
    if n_check > 0:
        indices = random.sample(range(len(sifted_alice)), n_check)
        errors = sum(sifted_alice[i] != sifted_bob[i] for i in indices)
        error_rate = errors / n_check
    else:
        error_rate = 0

    return {
        'n_sifted': len(sifted_alice),
        'error_rate': error_rate,
        'sifted_alice': sifted_alice,
        'sifted_bob': sifted_bob
    }

# Run without eavesdropping
result_clean = bb84_simulation(1000, eavesdrop=False)
print(f"No eavesdropping: {result_clean['n_sifted']} sifted bits, "
      f"error rate = {result_clean['error_rate']:.3f}")

# Run with eavesdropping
result_eve = bb84_simulation(1000, eavesdrop=True)
print(f"With eavesdropping: {result_eve['n_sifted']} sifted bits, "
      f"error rate = {result_eve['error_rate']:.3f}")
```

---

## 4.5 Security of BB84 🛡️

**Why Eve can't hide:**

1. If Eve measures in wrong basis, she introduces **50% error** in Bob's results when

Alice and Bob used same basis
2. Alice and Bob detect this when they compare a subset
3. If error rate > 0, they abort and try again

---

**Information-theoretic security:** The security is based on physics, not computational assumptions. Even with infinite computational power, Eve cannot break BB84 without being detected! 🔒

---

# Part 5: PennyLane Demos – Bell States and Measurements 💻

## 5.1 Creating and Measuring Bell States 🧪

```python
import pennylane as qml
import torch
import numpy as np
import matplotlib.pyplot as plt

# Create a 2-qubit device
dev = qml.device('default.qubit', wires=2)

@qml.qnode(dev, interface='torch')
def bell_state_circuit(state_idx=0):
    """Create one of the four Bell states"""

    # Prepare initial state based on index
    if state_idx == 0:      # |00⟩ -> |Φ⁺⟩
        pass
    elif state_idx == 1:    # |01⟩ -> |Ψ⁺⟩
        qml.PauliX(wires=1)
    elif state_idx == 2:    # |10⟩ -> |Φ⁻⟩
        qml.PauliX(wires=0)
    elif state_idx == 3:    # |11⟩ -> |Ψ⁻⟩
        qml.PauliX(wires=0)
        qml.PauliX(wires=1)

    # Create Bell state
    qml.Hadamard(wires=0)
    qml.CNOT(wires=[0, 1])

    # Measure both qubits in computational basis
    return qml.probs(wires=[0, 1])

@qml.qnode(dev, interface='torch')
def bell_measurement(state_idx=0):
    """Create Bell state and then measure in Bell basis"""

    # Create Bell state
    if state_idx == 0:      # |00⟩ -> |Φ⁺⟩
        pass
    elif state_idx == 1:    # |01⟩ -> |Ψ⁺⟩
        qml.PauliX(wires=1)
```

```
    elif state_idx == 2:    # |10⟩ -> |Φ⁻⟩
        qml.PauliX(wires=0)
    elif state_idx == 3:    # |11⟩ -> |Ψ⁻⟩
        qml.PauliX(wires=0)
        qml.PauliX(wires=1)

    qml.Hadamard(wires=0)
    qml.CNOT(wires=[0, 1])

    # Bell measurement (reverse the circuit)
    qml.CNOT(wires=[0, 1])
    qml.Hadamard(wires=0)

    # Measure in computational basis
    return qml.probs(wires=[0, 1])

# Demonstrate Bell states
bell_names = ['|Φ⁺⟩', '|Ψ⁺⟩', '|Φ⁻⟩', '|Ψ⁻⟩']

fig, axes = plt.subplots(1, 4, figsize=(16, 4))

for i in range(4):
    probs = bell_state_circuit(i).detach().numpy()

    axes[i].bar(range(4), probs)
    axes[i].set_xticks(range(4))
    axes[i].set_xticklabels(['00', '01', '10', '11'])
    axes[i].set_title(f'{bell_names[i]}')
    axes[i].set_ylim([0, 1])
    axes[i].set_ylabel('Probability')

plt.tight_layout()
plt.show()
```

## 5.2 Simulating Superdense Coding

```python
def superdense_coding(message_bits):
    """
    Simulate superdense coding
    message_bits: tuple (b0, b1) where b0,b1 ∈ {0,1}
    """

    dev = qml.device('default.qubit', wires=2)

    @qml.qnode(dev)
    def circuit(b0, b1):
        # Create Bell pair |Φ⁺⟩
        qml.Hadamard(wires=0)
        qml.CNOT(wires=[0, 1])

        # Alice encodes her message on her qubit (wires=0)
        if b0 == 1:  # First bit controls X
            qml.PauliX(wires=0)
        if b1 == 1:  # Second bit controls Z
            qml.PauliZ(wires=0)

        # Alice sends her qubit to Bob (in simulation, Bob now has both)

        # Bob performs Bell measurement
        qml.CNOT(wires=[0, 1])
        qml.Hadamard(wires=0)

        # Bob measures both qubits
        return qml.probs(wires=[0, 1])

    probs = circuit(message_bits[0], message_bits[1])
    measured_state = np.argmax(probs)

    # Convert measured state to bits
    b0_measured = measured_state // 2
    b1_measured = measured_state % 2

    return (b0_measured, b1_measured), probs

# Test all 4 messages
```

```python
messages = [(0,0), (0,1), (1,0), (1,1)]

for msg in messages:
    received, probs = superdense_coding(msg)
    print(f"Sent {msg} → Received {received} (probabilities: {probs.numpy().round(3)})")
```

## 5.3 Simulating Teleportation 📦

```python
def teleportation(alpha, beta):
    """
    Teleport an unknown state α|0⟩+β|1⟩
    alpha, beta: complex amplitudes with |α|²+|β|²=1
    """

    dev = qml.device('default.qubit', wires=3)

    @qml.qnode(dev)
    def circuit(alpha, beta):
        # Prepare unknown state on qubit 0
        qml.RY(2*np.arccos(alpha), wires=0)  # Simplified: assumes real α,β
        # For general state, would need more rotations

        # Create Bell pair on qubits 1 and 2
        qml.Hadamard(wires=1)
        qml.CNOT(wires=[1, 2])

        # Alice's operations
        qml.CNOT(wires=[0, 1])
        qml.Hadamard(wires=0)

        # Alice measures qubits 0 and 1 (we'll get the measurement outcomes)
        m0 = qml.measure(wires=0)
        m1 = qml.measure(wires=1)

        # Bob applies corrections based on Alice's results
        qml.cond(m1 == 1, qml.PauliX)(wires=2)
        qml.cond(m0 == 1, qml.PauliZ)(wires=2)

        # Return Bob's qubit state
        return qml.state(wires=2)

    teleported_state = circuit(alpha, beta)
    return teleported_state

# Test teleportation
alpha, beta = 0.6, 0.8  # 0.6|0⟩+0.8|1⟩
teleported = teleportation(alpha, beta)
```

```
print(f"Original amplitudes: ({alpha}, {beta})")
print(f"Teleported amplitudes: ({teleported[0]:.3f}, {teleported[1]:.3f})")
print(f"Fidelity: {abs(alpha*teleported[0].conj() + beta*teleported[1].conj())**2:.3f}")
```

---

# Part 6: Exercises for This Week 📝

## Exercise 1: Tensor Product Practice 🧩

Compute the following tensor products by hand:

1. $|0\rangle \otimes |+\rangle$
2. $X \otimes I$ applied to $|01\rangle$
3. CNOT applied to $|+0\rangle$ (where qubit 0 is control)

Verify your answers using NumPy's `np.kron`.

---

## Exercise 2: Bell State Identification 🔍

You are given one of the four Bell states but don't know which. Design a circuit that can identify it with certainty. Implement in PennyLane and test.

---

## Exercise 3: BB84 with Noise 🌫️

Modify the BB84 simulation to include:

- Realistic noise (e.g., 1% error rate even without Eve)
- Eve's optimal strategy (measure in random basis, resend)
- Error threshold for aborting (e.g., if error rate > 5%, abort)

Run simulations for different noise levels and determine the maximum noise that still allows secure key distribution.

## Exercise 4: Teleportation with Arbitrary States 🌀

Extend the teleportation simulation to handle arbitrary single-qubit states (with both amplitude and phase). Test by teleporting $|+i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$.

---

## Exercise 5: CHSH Inequality (Optional Challenge) 🏆

The CHSH inequality is a way to experimentally test that quantum mechanics cannot be explained by local hidden variables.

For the state $|\Phi^+\rangle$, measure:

- $\langle X \otimes X\rangle$, $\langle X \otimes Z\rangle$, $\langle Z \otimes X\rangle$, $\langle Z \otimes Z\rangle$

Compute the CHSH value:

$$S = \langle X \otimes X\rangle + \langle X \otimes Z\rangle + \langle Z \otimes X\rangle - \langle Z \otimes Z\rangle$$

For quantum mechanics, $S = 2\sqrt{2} \approx 2.828$. For classical theories, $|S| \leq 2$.

Implement this in PennyLane and verify the quantum value.

---

# Summary: Key Takeaways 🎯

## Math Concepts

- **Vectors** represent states, **operators** represent gates/measurements ➡
- **Inner products** give probabilities, **outer products** give projectors 📐
- **Tensor products** combine systems 🧩
- **Eigenvalues** are possible measurement outcomes 🔢
- **Unitary** matrices are reversible gates, **Hermitian** matrices are observables 🔄

## Quantum Communication

- **Entanglement** is a non-classical correlation that enables new protocols 🔗
- **Bell states** are the building blocks of quantum communication 🔔
- **Superdense coding** sends 2 bits with 1 qubit ✨
- **Teleportation** sends a qubit with 2 bits 📦
- **BB84** uses quantum mechanics for secure key distribution 🔐

## Why This Matters

These protocols aren't just theoretical curiosities:

- BB84 is implemented in real quantum networks 🌐
- Teleportation is used in quantum repeaters for long-distance communication 🔁
- Understanding entanglement is essential for quantum computing 💻

---

# Next Lecture Preview ⏩

**Week 4: Multi-Qubit Systems & Density Matrices**

- Partial trace and reduced density matrices
- Mixed states vs. pure states
- Quantifying entanglement
- Quantum channels and noise
- Open quantum systems

**Reading:** Nielsen & Chuang Chapter 2 📖

---

# AI Tool Demo for This Lecture 🤖

The code examples and explanations in this lecture were generated with the help of AI tool, DeepSeek-AI

**Remember:** Always verify AI-generated code by running it and understanding what each line does. The best way to learn is to modify the code and see what breaks! 💡

---

*"Entanglement is not one thing but a family of phenomena. The simplest form, the Bell state, is already enough to enable teleportation and superdense coding – two of the most stunning predictions of quantum information theory."* 📜